

说明

这篇文档总结了常见的非常规 Leetcode 类型的高频笔试题，每一道题目都有详细的解答。

目前共有下面这些问题的解答（后续会在[知识星球](#)里持续同步更新完善）：

1. 写三种单例模式的实现方式
2. 最后一人的编号
3. 交替打印奇偶数
4. 交替打印 ABC
5. 三个线程交替打印 1 到 99
6. 实现一个线程安全的计数器
7. 控制三个线程的执行顺序
8. 五人赛跑裁判
9. LRU 缓存实现
10. 用 Java 实现栈
11. 加权轮询算法的实现
12. 死锁
13. 快速排序
14. 生产者和消费者
15. 阻塞队列

本文档为 [JavaGuide 官方知识星球](#) 专属，后续会在我的[知识星球](#)里持续同步更新完善！

欢迎准备 Java 面试以及学习 Java 的同学加入我的 [知识星球](#)（点击链接即可查看星球的详细介绍），干货非常多，学习氛围也很不错！收费虽然是白菜价，但星球里的内容或许比你参加上万的培训班质量还要高。

JavaGuide官方知识星球 (限时优惠)

专属面试小册/一对一提问/简历修改
专属求职指南/不定时福利/学习打卡

— 点击图片即可详细了解 —

下面是星球提供的部分服务：

我的知识星球能为你提供什么？

努力做一个最优质的 Java 面试交流星球！加入到我的星球之后，你将获得：

1. 6 个高质量的专栏永久阅读，内容涵盖面试，源码解析，项目实战等内容！价值远超门票！
2. 多本原创 PDF 版本面试手册。
3. 免费的简历修改服务（已经累计帮助 4000+ 位球友修改简历）。
4. 一对一免费提问交流（专属建议，走心回答）。
5. 专属求职指南和建议，帮助你逆袭大厂！
6. 海量 Java 优质面试资源分享！价值远超门票！
7. 读书交流，学习交流，让我们共同努力创建一个纯粹的学习交流社区。
8. 不定期福利：节日抽奖、送书送课、球友线下聚会等等。
9.

其中的任何一项服务单独拎出来价值都远超星球门票了。

JavaGuide 官方网站地址：javaguide.cn。

写三种单例模式的实现方式

1、枚举（推荐）：

```
public enum Singleton {  
    INSTANCE;  
    public void doSomething(String str) {  
        System.out.println(str);  
    }  
}
```

《Effective Java》作者推荐的一种单例实现方式，简单高效，无需加锁，线程安全，可以避免通过反射破坏枚举单例。

2、静态内部类（推荐）：

```
public class Singleton {  
    // 私有化构造方法  
    private Singleton() {  
    }  
  
    // 对外提供获取实例的公共方法  
    public static Singleton getInstance() {  
        return SingletonInner.INSTANCE;  
    }  
  
    // 定义静态内部类  
    private static class SingletonInner{  
        private final static Singleton INSTANCE = new Singleton();  
    }  
}
```

当外部类 `Singleton` 被加载的时候，并不会创建静态内部类 `SingletonInner` 的实例对象。只有当调用 `getInstance()` 方法时，`SingletonInner` 才会被加载，这个时候才会创建单例对象 `INSTANCE`。 `INSTANCE` 的唯一性、创建过程的线程安全性，都由 JVM 来保证。

这种方式同样简单高效，无需加锁，线程安全，并且支持延时加载。

3、双重校验锁：

```
public class Singleton {  
  
    private volatile static Singleton uniqueInstance;  
  
    // 私有化构造方法  
    private Singleton() {  
    }  
  
    public static Singleton getUniqueInstance() {  
        //先判断对象是否已经实例过，没有实例化过才进入加锁代码  
        if (uniqueInstance == null) {  
            //类对象加锁  
            synchronized (Singleton.class) {  
                if (uniqueInstance == null) {  
                    uniqueInstance = new Singleton();  
                }  
            }  
        }  
        return uniqueInstance;  
    }  
}
```

`uniqueInstance` 采用 `volatile` 关键字修饰也是很有必要的，`uniqueInstance = new Singleton();` 这段代码其实是分为三步执行：

1. 为 `uniqueInstance` 分配内存空间
2. 初始化 `uniqueInstance`
3. 将 `uniqueInstance` 指向分配的内存地址

但是由于 JVM 具有指令重排的特性，执行顺序有可能变成 1->3->2。指令重排在单线程环境下不会出现问题，但是在多线程环境下会导致一个线程获得还没有初始化的实例。例如，线程 T1 执行了 1 和 3，此时 T2 调用 `getUniqueInstance ()` 后发现 `uniqueInstance` 不为空，因此返回 `uniqueInstance`，但此时 `uniqueInstance` 还未被初始化。

这种方式实现起来较麻烦，但同样线程安全，支持延时加载。

推荐阅读：[Java 并发常见面试题总结（中）](#)。

最后一人的编号

问题描述：编号为 1-n 的循环报 1-3，报道 3 的出列，求最后一人的编号

标准的约瑟夫环问题。有 n 个人围成一个圈，从某个人开始报数，报到某个特定数字（本题中为 3）时该人出圈，直到只剩下一个人为止。

解决约瑟夫环问题，可以分两种情况：

1. 我们要求出最后留下的那个人的编号（本题要求）。
2. 求全过程，即要算出每轮出局的人。

有多种方法可以解决约瑟夫环问题，其中一种是使用递归的方式。

本题的约瑟夫环问题的公式为： $(f(n - 1, k) + k - 1) \% n + 1$ 。f(n,k) 表示 n 个人报数，每次报数报到 k 的人出局，最终最后一个人的编号。

假设 n 为 10，k 为 3，逆推过程如下：

- $f(1, 3) = 1$ （当 $n = 1$ 时，只有一个人，最后一人的编号就为 1）；
- $f(2, 3) = (f(1, 3) + 3 - 1) \% 2 + 1 = 3 \% 2 + 1 = 2$ （当 $n = 2$ 时，最后一人的编号为 2）；
- $f(3, 3) = (f(2, 3) + 3 - 1) \% 3 + 1 = 4 \% 3 + 1 = 2$ （当 $n = 3$ 时，最后一人的编号为 2）；
- $f(4, 3) = (f(3, 3) + 3 - 1) \% 4 + 1 = 4 \% 4 + 1 = 1$ （当 $n = 4$ 时，最后一人的编号为 1）；
- ...
- $f(10, 3) = 3$ （当 $n = 10$ 时，最后一人的编号为 4）；

这个问题对应剑指 Offer 62. 圆圈中最后剩下的数字，两者意思是类似的，比较简单。

```
public class Josephus {

    // 定义递归函数
    public static int f(int n, int k) {
        // 如果只有一个人，则返回 1
        if (n == 1) {
            return 1;
        }
        return (f(n - 1, k) + k - 1) % n + 1;
    }

    public static void main(String[] args) {
        int n = 10;
        int k = 3;
        System.out.println("最后留下的那个人的编号是: " + f(n, k));
    }
}
```

输出：

```
最后留下的那个人的编号是: 4
```

两个线程交替打印奇偶数

问题描述：写两个线程打印 1-100，一个线程打印奇数，一个线程打印偶数。

这道题的实现方式还是挺多的，线程的等待/通知机制（`wait()` 和 `notify()`）、信号量 `Semaphore` 等都可以实现。

synchronized+wait/notify 实现

我们先定义一个类 `ParityPrinter` 用于打印奇数和偶数。

```
public class ParityPrinter {  
    private final int max;  
    // 从1开始计数  
    private int count = 1;  
    private final Object lock = new Object();  
  
    public ParityPrinter(int max) {  
        this.max = max;  
    }  
  
    public void printOdd() {  
        print(true);  
    }  
  
    public void printEven() {  
        print(false);  
    }  
  
    private void print(boolean isOdd) {  
        for (int i = 1; i <= max; i += 2) {  
            // 确保同一时间只有一个线程可以执行内部代码块  
            synchronized (lock) {  
                // 等待直到轮到当前线程打印  
                // count为奇数时奇数线程打印, count为偶数时偶数线程打印  
                while (isOdd == (count % 2 == 0)) {  
                    try {  
                        lock.wait();  
                    } catch (InterruptedException e) {  
                        Thread.currentThread().interrupt();  
                        return;  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        System.out.println(Thread.currentThread().getName() + " :
" + count++);

        // 通知等待的线程
        lock.notify();
    }
}
}
}
}

```

ParityPrinter 类中的变量和方法介绍：

- `max` : 最大打印数值，由构造函数传入。
- `count` : 从 1 开始的计数器，用于追踪当前打印到的数字。
- `lock` : 一个对象锁，用于线程间的同步控制。
- `printOdd()` 和 `printEven()` : 分别启动打印奇数和偶数的逻辑，实际上调用了私有的 `print()` 方法，并传入线程名称前缀和一个布尔值表示打印奇数(`true`)还是偶数(`false`)。

接着，我们创建两个线程，一个负责打印奇数，一个负责打印偶数。

```

// 打印 1-100
ParityPrinter printer = new ParityPrinter(100);
// 创建打印奇数和偶数的线程
Thread t1 = new Thread(printer::printOdd, "Odd");
Thread t2 = new Thread(printer::printEven, "Even");
t1.start();
t2.start();

```

输出：


```
Odd : 1
Even : 2
Odd : 3
Even : 4
Odd : 5
...
Odd : 95
Even : 96
Odd : 97
Even : 98
Odd : 99
Even : 100
```

Semaphore 实现

如果想要把上面的代码修改为基于 Semaphore 实现也挺简单的。

```
public class ParityPrinter {
    private final int max;
    private int count = 1;
    // 初始为1, 奇数线程先获取
    private final Semaphore oddSemaphore = new Semaphore(1);
    // 初始为0, 偶数线程等待
    private final Semaphore evenSemaphore = new Semaphore(0);

    public ParityPrinter(int max) {
        this.max = max;
    }

    public void printOdd() {
        print(oddSemaphore, evenSemaphore);
    }

    public void printEven() {
```

```

        print(evenSemaphore, oddSemaphore);
    }

    private void print(Semaphore currentSemaphore, Semaphore
nextSemaphore) {
        for (int i = 1; i <= max; i += 2) {
            try {
                // 获取当前信号量
                currentSemaphore.acquire();

                System.out.println(Thread.currentThread().getName() + " :
" + count++);

                // 释放下一个信号量
                nextSemaphore.release();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}

```

可以看到，我们这里使用两个信号量 `oddSemaphore` 和 `evenSemaphore` 来确保两个线程交替执行。`oddSemaphore` 信号量先获取，也就是先执行奇数输出。一个线程执行完之后，就释放下一个信号量。

三个线程交替打印 ABC

问题描述：写三个线程打印 "ABC"，一个线程打印 A，一个线程打印 B，一个线程打印 C，一共打印 10 轮。

这个问题其实和上面的交替打印奇偶数是一样的。我们这里提供一个 `Semaphore` 版本和 `ReentrantLock` + `Condition` 版本。

Semaphore 实现

我们先定义一个类 `ABCPrinter` 用于实现三个线程交替打印 ABC。

```
public class ABCPrinter {
    private final int max;
    // 从线程 A 开始执行
    private final Semaphore semaphoreA = new Semaphore(1);
    private final Semaphore semaphoreB = new Semaphore(0);
    private final Semaphore semaphoreC = new Semaphore(0);

    public ABCPrinter(int max) {
        this.max = max;
    }

    public void printA() {
        print("A", semaphoreA, semaphoreB);
    }

    public void printB() {
        print("B", semaphoreB, semaphoreC);
    }

    public void printC() {
        print("C", semaphoreC, semaphoreA);
    }

    private void print(String alphabet, Semaphore currentSemaphore,
        Semaphore nextSemaphore) {
        for (int i = 1; i <= max; i++) {
            try {
                currentSemaphore.acquire();
                System.out.println(Thread.currentThread().getName() + " : " + alphabet);
                // 传递信号给下一个线程
                nextSemaphore.release();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
```

```
    }  
    }  
}  
}
```

可以看到，我们这里用到了三个信号量，分别用于控制这三个线程的交替执行。`semaphoreA` 信号量先获取，也就是先输出“A”。一个线程执行完之后，就释放下一个信号量。也就是，A 线程执行完之后释放 `semaphoreB` 信号量，B 线程执行完之后释放 `semaphoreC` 信号量，以此类推。

接着，我们创建三个线程，分别用于打印 ABC。

```
ABCPrinter printer = new ABCPrinter(10);  
Thread t1 = new Thread(printer::printA, "Thread A");  
Thread t2 = new Thread(printer::printB, "Thread B");  
Thread t3 = new Thread(printer::printC, "Thread C");  
  
t1.start();  
t2.start();  
t3.start();
```

输出如下：

```
Thread A : A  
Thread B : B  
Thread C : C  
.....  
Thread A : A  
Thread B : B  
Thread C : C
```

ReentrantLock + Condition 实现

思路和 `synchronized` + `wait/notify` 很像。

```
public class ABCPrinter {
    private final int max;

    // 用来指示当前应该打印的线程序号, 0-A, 1-B, 2-C
    private int turn = 0;

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition conditionA = lock.newCondition();
    private final Condition conditionB = lock.newCondition();
    private final Condition conditionC = lock.newCondition();

    public ABCPrinter(int max) {
        this.max = max;
    }

    public void printA() {
        print("A", conditionA, conditionB);
    }

    public void printB() {
        print("B", conditionB, conditionC);
    }

    public void printC() {
        print("C", conditionC, conditionA);
    }

    private void print(String name, Condition currentCondition, Condition
nextCondition) {
        for (int i = 0; i < max; i++) {
            lock.lock();

            try {
                // 等待直到轮到当前线程打印
            }
        }
    }
}
```

```

        // turn 变量的值需要与线程要打印的字符相对应，例如，如果turn是0，
        且当前线程应该打印"A"，则条件满足。如果不满足，当前线程调用
        currentCondition.await()进入等待状态。

        while (!((turn == 0 && name.charAt(0) == 'A') || (turn ==
1 && name.charAt(0) == 'B') || (turn == 2 && name.charAt(0) == 'C')) {
            currentCondition.await();
        }
        System.out.println(Thread.currentThread().getName() + " :
" + name);

        // 更新打印轮次，并唤醒下一个线程
        turn = (turn + 1) % 3;
        nextCondition.signal();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        lock.unlock();
    }
}
}
}
}
}

```

在上面的代码中，三个线程的协调主要依赖：

- `ReentrantLock lock`：用于线程同步的可重入锁，确保同一时刻只有一个线程能修改共享资源。
- `Condition conditionA/B/C`：分别与"A"、"B"、"C"线程关联的条件变量，用于线程间的协调通信。一个线程执行完之后，通过调用 `nextCondition.signal()` 唤醒下一个应该打印的线程。

三个线程交替打印 1 到 99

问题描述：写三个线程 A、B、C，A 线程打印 $3n+1$ ，B 线程打印 $3n+2$ ，C 线程打印 $3n$ 。

这道题和三个线程交替打印 ABC 这道题有挺多相似之处，这里我们分享一种 `ReentrantLock` + `Condition` 的实现。

```

public class NumberPrinter {
    private final int max;
    private int number = 1;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition conditionForMultiplesOfThree =
lock.newCondition();
    private final Condition conditionForForm3nPlus1 =
lock.newCondition();
    private final Condition conditionForForm3nPlus2 =
lock.newCondition();

    public NumberPrinter(int max) {
        this.max = max;
    }

    // 打印 3n
    public void print3n() {
        printNumber(() -> number % 3 == 0, conditionForMultiplesOfThree,
conditionForForm3nPlus1);
    }

    // 打印 3n+1
    public void print3nPlus1() {
        printNumber(() -> number % 3 == 1, conditionForForm3nPlus1,
conditionForForm3nPlus2);
    }

    // 打印 3n+2
    public void print3nPlus2() {
        printNumber(() -> number % 3 == 2, conditionForForm3nPlus2,
conditionForMultiplesOfThree);
    }

    private void printNumber(java.util.function.Supplier<Boolean>
shouldPrint, Condition currentCondition, Condition nextCondition) {
        while (number < max - 1) {
            lock.lock();

```

```

        try {
            while (!shouldPrint.get()) {
                currentCondition.await();
            }
            System.out.println(Thread.currentThread().getName() + " :
" + number++);
            nextCondition.signal();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            lock.unlock();
        }
    }
}
}

```

代码逻辑比较简单，需要注意的是，我们这里使用了 `Supplier<Boolean>` 匿名函数来判断当前数字是否符合打印条件。

接着，我们创建三个线程，分别用于打印对应的数字。

```

NumberPrinter printer = new NumberPrinter(99);
Thread t1 = new Thread(printer::print3nPlus1, "Thread A");
Thread t2 = new Thread(printer::print3nPlus2, "Thread B");
Thread t3 = new Thread(printer::print3n, "Thread C");

t1.start();
t2.start();
t3.start();

```

输出：


```
Thread A : 1
Thread B : 2
Thread C : 3
.....
Thread A : 94
Thread B : 95
Thread C : 96
Thread A : 97
Thread B : 98
Thread C : 99
```

实现一个线程安全的计数器

问题描述：实现一个线程安全的计数器，100 个线程，每个线程累加 100 次

`AtomicLong` 通过使用 CAS（Compare-And-Swap）操作，实现了无锁的线程安全机制，能够对长整型数据进行原子操作。高并发的场景下，乐观锁相比悲观锁来说，不存在锁竞争造成线程阻塞，也不会有死锁的问题，在性能上往往会更胜一筹。

```
// 创建一个线程安全的计数器
AtomicLong counter = new AtomicLong();

// 创建一个固定大小的线程池（演示使用，推荐手动创建线程池）
ExecutorService executor = Executors.newFixedThreadPool(100);

for (int i = 0; i < 100; i++) {
    // 100 个线程，每个线程累加 100 次
    executor.submit(() -> {
        for (int i1 = 0; i1 < 100; i1++) {
            counter.incrementAndGet();
        }
    });
}

// 关闭线程池，等待所有任务完成
executor.shutdown();
```

```
executor.awaitTermination(Long.MAX_VALUE, TimeUnit.NANOSECONDS);

// 输出最终计数器的值

System.out.println("Final Counter Value: " + counter.get());
```

输出：

10000

虽然 `AtomicLong` 的性能已经相当优秀，但在高并发场景下仍存在一些效率问题。JDK 8 新增了一个原子性递增或者递减类 `LongAdder` 用来克服在高并发下使用 `AtomicLong` 的一些缺点。

使用 `LongAdder` 改造后的代码如下：

```
LongAdder counter = new LongAdder();

ExecutorService executor = Executors.newFixedThreadPool(100);

for (int i = 0; i < 100; i++) {
    executor.submit(() -> {
        for (int i1 = 0; i1 < 100; i1++) {
            counter.increment();
        }
    });
}

executor.shutdown();

executor.awaitTermination(Long.MAX_VALUE, TimeUnit.NANOSECONDS);

System.out.println("Final Counter Value: " + counter.sum());
```

`LongAdder` 使用 `increment()` 方法累加，所有累加的总和通过 `sum()` 方法获取。

控制三个线程的执行顺序

问题描述：假设有 T1、T2、T3 三个线程，你怎样保证 T2 在 T1 执行完后执行，T3 在 T2 执行完后执行？

这道题不难，网上大部分人都是用 `join()` 或者 `CountDownLatch` 实现，但个人更推荐 `CompletableFuture`。

代码如下（这里为了简化代码，用到了 Hutool 的线程工具类 `ThreadUtil` 和日期时间工具类 `DateUtil`）：

```
// 创建一个固定大小的线程池
ExecutorService executor = Executors.newFixedThreadPool(3);

CompletableFuture<Void> futureT1 = CompletableFuture.runAsync(() -> {
    System.out.println("T1 is executing in thread: " +
        Thread.currentThread().getName() + ". Current time: " + DateUtil.now());
    ThreadUtil.sleep(1000);
}, executor);

CompletableFuture<Void> futureT2 = futureT1.thenRunAsync(() -> {
    System.out.println("T2 is executing in thread: " +
        Thread.currentThread().getName() + " after T1. Current time: " +
        DateUtil.now());
    ThreadUtil.sleep(1000);
}, executor);

CompletableFuture<Void> futureT3 = futureT2.thenRunAsync(() -> {
    System.out.println("T3 is executing in thread: " +
        Thread.currentThread().getName() + " after T2. Current time: " +
        DateUtil.now());
    ThreadUtil.sleep(1000);
}, executor);

ThreadUtil.sleep(3000);
```

可以看到，我们这里通过 `thenRunAsync()` 方法就实现了 T1、T2、T3 的顺序执行。

`thenRunAsync()` 方法的作用就是做完第一个任务后，再做第二个任务。也就是说某个任务执行完成后，执行回调方法

输出：

```
T1 is executing in thread: pool-1-thread-1. Current time: 2024-07-03
10:04:28
T2 is executing in thread: pool-1-thread-2 after T1. Current time: 2024-
07-03 10:04:29
T3 is executing in thread: pool-1-thread-3 after T2. Current time: 2024-
07-03 10:04:30
```

⚠ 注意：`CompletableFuture` 默认使用 `ForkJoinPool.commonPool()` 作为线程池来执行异步任务。如果多个任务连续执行，可能会被分配到同一个线程。要确保任务在不同线程中执行，可以传递自定义的 `Executor` 给 `runAsync` 和 `thenRunAsync` 方法。

如果我们想要实现 T3 在 T2 和 T1 执行完后执行，T2 和 T1 可以同时执行,应该怎么办呢？

```
// T1
CompletableFuture<Void> futureT1 = CompletableFuture.runAsync(() -> {
    System.out.println("T1 is executing. Current time: " +
DateUtil.now());
    // 模拟耗时操作
    ThreadUtil.sleep(1000);
});

// T2
CompletableFuture<Void> futureT2 = CompletableFuture.runAsync(() -> {
    System.out.println("T2 is executing. Current time: " +
DateUtil.now());
    ThreadUtil.sleep(1000);
});
```

```
// 使用allOf()方法合并T1和T2的CompletableFuture, 等待它们都完成
CompletableFuture<Void> bothCompleted = CompletableFuture.allOf(futureT1,
futureT2);
// 当T1和T2都完成后, 执行T3
bothCompleted.thenRunAsync(() -> System.out.println("T3 is executing
after T1 and T2 have completed.Current time: " + DateUtil.now()));
// 等待所有任务完成, 验证效果
ThreadUtil.sleep(3000);
```

同样非常简单, 可以通过 `CompletableFuture` 的 `allOf()` 这个静态方法来并行运行多个 `CompletableFuture`。然后, 再利用 `thenRunAsync()` 方法即可。

这个问题还有非常多的扩展, 上面提到的场景都是异步任务编排最简单的场景。

`CompletableFuture` 的详细介绍请看我写的这篇文章: [CompletableFuture 详解](#)。

JavaGuide官方知识星球 (限时优惠)

专属面试小册/一对一提问/简历修改
专属求职指南/不定时福利/学习打卡

— 点击图片即可详细了解 —

五人赛跑裁判

问题描述: 有 5 个人赛跑, 请你设计一个多线程的裁判程序给出他们赛跑的结果顺序, 5 个人的速度随机处理。

我们借助线程池和 `CountDownLatch` 来实现这一需求即可。

```
// 使用 AtomicInteger 确保线程安全
```

```
public static AtomicInteger num = new AtomicInteger(0);

public static String[] res = new String[5];

private static final int threadCount = 5;

public static void main(String[] args) throws InterruptedException {
    // 创建一个固定大小的线程池（演示使用，推荐手动创建线程池）

    ExecutorService threadPool =
Executors.newFixedThreadPool(threadCount);

    final CountDownLatch countDownLatch = new
CountDownLatch(threadCount);

    Random random = new Random();

    for (int i = 0; i < threadCount; i++) {
        final int threadnum = i;

        threadPool.execute(() -> {
            try {
                // 模拟随机耗时

                int sleepTime = random.nextInt(401) + 100; // 100ms ~
500ms

                Thread.sleep(sleepTime);

                // 使用 AtomicInteger 确保线程安全

                int index = num.getAndIncrement();
                res[index] = "运动员" + (threadnum + 1) + "消耗的时间为" +
sleepTime;

                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                } finally {
                    countDownLatch.countDown();
                }
            });
        }

        // 等待所有线程完成

        countDownLatch.await();

        threadPool.shutdown();
    }
}
```

```
// 输出结果  
for (String re : res) {  
    System.out.println(re);  
}  
}
```

输出：

```
运动员1消耗的时间为111  
运动员4消耗的时间为252  
运动员5消耗的时间为300  
运动员2消耗的时间为415  
运动员3消耗的时间为424
```

解题的核心是 `AtomicInteger` 和 `CountDownLatch` 类的运用：

- `AtomicInteger` 是一个线程安全的整数类，支持原子性操作。
- `CountDownLatch` 是一个线程同步工具类，用于让主线程等待其他线程完成工作。在本题中，初始化计数器为线程数 5 `new CountDownLatch(5)`。每个线程完成任务后，调用 `countDownLatch.countDown()`，主线程调用 `countDownLatch.await()`。

完整的执行流程如下：

1. 创建一个固定大小的线程池和一个 `CountDownLatch`，初始化为5。
2. 提交5个线程任务到线程池，模拟每个运动员完成比赛的过程：
 - 每个线程随机等待一定时间（100ms~500ms），表示运动员比赛时长。
 - 使用 `AtomicInteger` 确保线程安全，将比赛结果写入数组。
 - 每个线程完成后，调用 `countDown()` 减少计数器值。
3. 主线程调用 `await()`，等待所有线程完成。
4. 所有线程完成后，主线程输出比赛结果。

5. 关闭线程池。

LRU 缓存实现

如果碰到这种题目先不要慌张，现在脑海里回忆一遍 LRU 的基本概念：**LRU (Least Recently Used, 最近最少使用)** 是一种缓存算法，其核心思想是**将最近最少使用的缓存项移除**，以便为更常用的缓存项腾出空间。

适用场景：

- 频繁访问：LRU 算法适用于那些有频繁访问的数据，比如缓存、页面置换等场景。
- 有局部性：当访问模式具有局部性，即近期访问的数据更可能在未来被再次访问时，LRU 算法能够有较好的表现。
- 数据访问分布均匀：如果数据的访问分布较为均匀，没有出现热点数据或周期性访问模式，LRU 算法的命中率较高。
- 缓存容量适中：LRU 算法适用于缓存容量适中的场景，过大的缓存可能导致淘汰开销增大，而过小的缓存则可能导致频繁缓存失效。

在 Java 中，可以使用 `LinkedHashMap` 来实现 LRU 缓存。使用 `LinkedHashMap` 实现 LRU 缓存可以极大地简化代码，因为 `LinkedHashMap` 已经内置了按照访问顺序排序的功能。所以使用 `LinkedHashMap` 确实可以避免手动实现双向链表和节点的逻辑。

为了使用 `LinkedHashMap` 来实现 LRU 缓存，在创建 `LinkedHashMap` 对象时设置它的访问顺序为 `true`，这样元素将按照**访问顺序**进行排序。然后，我们可以重写它的 `removeEldestEntry` 方法来控制是否移除最老的数据。

```
import java.util.LinkedHashMap;
import java.util.Map;

public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true);
    }
}
```



```
        this.capacity = capacity;
    }

    public int get(int key) {
        return super.getDefault(key, -1);
    }

    public void put(int key, int value) {
        super.put(key, value);
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        // 当缓存元素个数超过容量时，移除最老的元素
        return size() > capacity;
    }

    public static void main(String[] args) {
        // 创建一个容量为3的LRU缓存
        LRUCache<Integer, String> lruCache = new LRUCache<>(3);

        // 添加数据
        lruCache.put(1, "One");
        lruCache.put(2, "Two");
        lruCache.put(3, "Three");

        // 此时缓存为: {1=One, 2=Two, 3=Three}

        // 访问某个元素，使其成为最近访问的元素
        String value = lruCache.get(2);

        // 此时缓存为: {1=One, 3=Three, 2=Two}

        // 添加新的数据，触发淘汰
        lruCache.put(4, "Four");
    }
}
```

```
// 此时缓存为: {3=Three, 2=Two, 4=Four}
// 元素1被淘汰, 因为它是最近最少访问的元素
}
}
```

在上面的代码中, `removeEldestEntry` 方法是用于控制是否移除最老的数据。当缓存大小超过指定容量时, `removeEldestEntry` 会返回 `true`, 表示需要移除最老的数据。这样, 通过 `LinkedHashMap` 和重写 `removeEldestEntry` 方法, 实现了一个简单的 LRU 缓存。

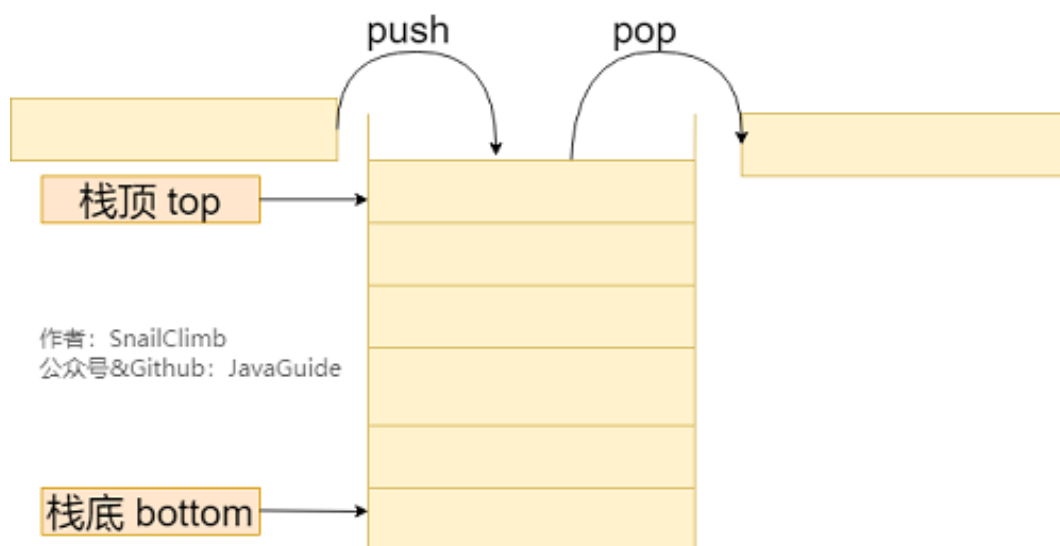
对于面试时遇到 LRU 实现的问题, 如果不限使用特定的数据结构, 可以直接采用上述方案来进行简单实现。

关于 `LinkedHashMap` 的详细介绍, 推荐你看看这篇: [LinkedHashMap 源码分析](#)。

用 Java 实现栈

同样是实现某某功能, 那么还是来回顾一下它的概念, 这样能快速在脑海里构建出雏形思路。

先来看一下栈的示意图:



然后回忆一下栈中的一些操作方法:

- `push(element)` : 将元素压入栈顶。
- `pop()` : 弹出并返回栈顶的元素。
- `top()` : 返回栈顶的元素, 但不删除。
- `isEmpty()` : 判断栈是否为空。
- `size()` : 返回栈中元素的个数。

当实现这些操作方法时, 需要考虑边界情况, 如栈为空时执行 `pop()` 或 `top()` 操作的处理。

下面来简单实现一下:

1. 首先定义一个 `Stack` 类;
2. 在 `Stack` 类中声明私有变量, 用于存储栈的元素。可以使用数组或者链表来实现栈的底层数据结构, 这里以数组为例。
3. 实现构造方法, 初始化栈的大小和其他必要的变量。
4. 实现上述的栈的操作方法

```
class Stack {  
    private int[] arr;  
    private int top;  
  
    public Stack(int capacity) {  
        arr = new int[capacity];  
        top = -1;  
    }  
  
    public void push(int element) {  
        if (top == arr.length - 1) {  
            throw new IllegalStateException("栈已满!");  
        }  
        top++;  
        arr[top] = element;  
    }  
}
```

```
public int pop() {
    if (isEmpty()) {
        throw new IllegalStateException("栈为空!");
    }
    int element = arr[top];
    top--;
    return element;
}

public int top() {
    if (isEmpty()) {
        throw new IllegalStateException("栈为空!");
    }
    return arr[top];
}

public boolean isEmpty() {
    return top == -1;
}

public int size() {
    return top + 1;
}
}
```

在上述代码中，我们使用数组 `arr` 存储栈的元素，使用 `top` 变量记录栈顶的索引。

在 `push()` 方法中，首先检查栈是否已满，如果已满则抛出异常；否则将元素插入到 `top` 索引处，并将 `top` 值加 1。

在 `pop()` 方法中，首先检查栈是否为空，如果为空则抛出异常；否则从 `top` 索引处取出元素，然后将 `top` 减 1。

在面试过程中，编写代码之前可以先和面试官讨论栈的基本操作和边界情况，然后再进行代码编写。同时，确保代码的可读性和注释的适当添加，以便面试官更好地理解你的代码。

更多关于栈的介绍，推荐阅读[线性数据结构详解](#)这篇文章。

加权轮询算法的实现

首先还是先来看一下概念：加权轮询算法是一种常用的负载均衡算法，广泛应用于分布式系统中。它通过给不同的服务器分配不同的权重来实现请求的均衡分发。

该算法的运行过程如下：

1. 算法开始时，为每个服务器分配一个**初始权重**，通常情况下，权重值可以根据服务器的性能、处理能力、连接数等因素进行设置。权重值越高，表示该服务器可以处理更多的请求。
2. 当有新的请求到达时，负载均衡器会选择具有最高权重的服务器处理该请求。
3. 选定服务器处理请求后，该服务器的权重会减去一个事先设定好的值，以降低其权重，使得在下一次轮询中被选中的可能性减小。
4. 当所有服务器的权重值都减为 0 时，它们将重新被赋予初始权重值，然后再次参与请求的分发。
5. 重复上述步骤，直到服务终止或者负载均衡器收到终止信号。

加权轮询算法的优点是：简单且易于理解，它能够根据服务器的处理能力自动调整请求的分发比例。较为强大的服务器可以拥有更高的权重，从而更频繁地接收到请求，以提高整个系统的吞吐量。此外，该算法不需要记录每个会话的状态信息，因此具有较低的空间复杂度。

然而，加权轮询算法也存在一些缺点：首先，它并不能考虑服务器的实际负载情况，只是根据预设的权重进行分发，无法根据服务器的实时负载状态进行动态调整。另外，如果权重的设置不合理，可能导致某些服务器负载过重，而其他服务器处于闲置状态。

根据上面的介绍，简单概括：使用加权轮询（Weighted Round Robin）算法时，服务器根据每个服务器的权重值来决定请求应该发送到哪个服务器。权重越高的服务器将获得更多的请求。

下面用 Java 来简单实现一下。

加权轮询核心类 WeightedRoundRobin :

```
class WeightedRoundRobin {  
    private final List<Server> servers;  
    private int currentIndex;  
    private int maxWeight;  
    private int gcdWeight;  
  
    public WeightedRoundRobin(List<Server> servers) {  
        this.servers = new ArrayList<>(servers);  
        currentIndex = -1;  
        if (!servers.isEmpty()) {  
            this.maxWeight = findMaxWeight();  
            this.gcdWeight = calculateGcd();  
        }  
    }  
  
    /**  
     * 按照加权轮询逻辑检索下一个服务器。  
     */  
    public Server getNextServer() {  
        if (servers.isEmpty()) {  
            // 没有服务器可用  
            return null;  
        }  
  
        while (true) {  
            currentIndex = (currentIndex + 1) % servers.size();  
            if (currentIndex == 0) {  
                maxWeight -= gcdWeight;  
                if (maxWeight <= 0) {  
                    maxWeight = findMaxWeight();  
                    if (maxWeight == 0) {  
                        // 所有服务器的权重都为0，无法分配请求  
                        return null;  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        }

        }

    }

    if (servers.get(currentIndex).getWeight() >= maxWeight) {
        return servers.get(currentIndex);
    }

}

/**
 * 查找所有服务器中的最大权重。
 */
private int findMaxWeight() {
    return
servers.stream().mapToInt(Server::getWeight).max().orElse(0);
}

/**
 * 计算所有服务器权重的最大公约数。
 */
private int calculateGcd() {
    return
servers.stream().mapToInt(Server::getWeight).reduce(this::gcd).orElse(0);
}

/**
 * 计算两个数的最大公约数。
 */
private int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

```

```
}
```

服务器对象：

```
class Server {  
    private String name;  
    private int weight;  
  
    public Server(String name, int weight) {  
        this.name = name;  
        this.weight = weight;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getWeight() {  
        return weight;  
    }  
}
```

接着，我们写段代码实际测试一下效果。

```
Server server1 = new Server("Server1", 1);  
Server server2 = new Server("Server2", 2);  
Server server3 = new Server("Server3", 3);  
List<Server> servers = Arrays.asList(server1, server2, server3);  
WeightedRoundRobin wrw = new WeightedRoundRobin(servers);  
  
// 记录选中次数  
int count1 = 0, count2 = 0, count3 = 0;  
int totalCalls = 12000; // 大量次数以检验权重比例
```



```

for (int i = 0; i < totalCalls; i++) {
    Server selected = wrr.getNextServer();
    switch (selected.getName()) {
        case "Server1":
            count1++;
            break;
        case "Server2":
            count2++;
            break;
        case "Server3":
            count3++;
            break;
    }
}

System.out.println("Server1:" + count1 + ",Server2:" + count2 +
    ",Server3:" + count3);

```

在上述示例中，我们创建了三个服务器对象，并将它们的权重分别设置为 1、2 和 3。然后创建 `WeightedRoundRobin` 对象，传入服务器列表。最后模拟了 12000 个请求的分发，并打印每个请求被发送到的服务器。

输出：

```
Server1:2000,Server2:4000,Server3:6000
```

可以看到输出结果是符合预期的。

死锁

无论在笔试还是面试中遇到死锁问题，都要第一时间跳出死锁的概念，注意抓住重点回答。

什么是死锁：

线程死锁描述的是这样一种情况：多个线程同时被阻塞，它们中的一个或者全部都在等待某个资源被释放。由于线程被无限期地阻塞，因此程序不可能正常终止。

如下图所示，线程 A 持有资源 2，线程 B 持有资源 1，他们同时都想申请对方的资源，所以这两个线程就会互相等待而进入死锁状态。



下面通过一个例子来说明线程死锁,代码模拟了上图的死锁的情况 (代码来源于《并发编程之美》):

```
public class DeadLockDemo {
    private static Object resource1 = new Object();//资源 1
    private static Object resource2 = new Object();//资源 2

    public static void main(String[] args) {
        new Thread(() -> {
            synchronized (resource1) {
                System.out.println(Thread.currentThread() + "get
resource1");

                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                System.out.println(Thread.currentThread() + "waiting get
resource2");

                synchronized (resource2) {
                    System.out.println(Thread.currentThread() + "get
resource2");
                }
            }
        }, "线程 1").start();

        new Thread(() -> {
```

```

        synchronized (resource2) {
            System.out.println(Thread.currentThread() + "get
resource2");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println(Thread.currentThread() + "waiting get
resource1");
            synchronized (resource1) {
                System.out.println(Thread.currentThread() + "get
resource1");
            }
        }
    }, "线程 2").start();
}
}

```

输出：

```

Thread[线程 1,5,main]get resource1
Thread[线程 2,5,main]get resource2
Thread[线程 1,5,main]waiting get resource2
Thread[线程 2,5,main]waiting get resource1

```

线程 A 通过 `synchronized (resource1)` 获得 `resource1` 的监视器锁，然后通过 `Thread.sleep(1000);` 让线程 A 休眠 1s 为的是让线程 B 得到执行然后获取到 `resource2` 的监视器锁。线程 A 和线程 B 休眠结束了都开始企图请求获取对方的资源，然后这两个线程就会陷入互相等待的状态，这也就产生了死锁。

上面的例子符合产生死锁的四个必要条件：

1. 互斥条件：该资源任意一个时刻只由一个线程占用。
2. 请求与保持条件：一个线程因请求资源而阻塞时，对已获得的资源保持不放。
3. 不剥夺条件：线程已获得的资源在未使用完之前不能被其他线程强行剥夺，只有自己使用完毕后才释放资源。
4. 循环等待条件：若干线程之间形成一种头尾相接的循环等待资源关系。

如何防和避免死锁：

反着来就行了，破坏死锁的产生的必要条件即可：

1. 破坏请求与保持条件：一次性申请所有的资源。
2. 破坏不剥夺条件：占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。
3. 破坏循环等待条件：靠按序申请资源来预防。按某一顺序申请资源，释放资源则反序释放。破坏循环等待条件。

如果在笔试中，上面的代码写出来后，要求改动，改成不形成死锁---->

避免死锁就是在资源分配时，借助于算法（比如银行家算法）对资源分配进行计算评估，使其进入安全状态。

安全状态 指的是系统能够按照某种线程推进顺序（P1、P2、P3.....Pn）来为每个线程分配所需资源，直到满足每个线程对资源的最大需求，使每个线程都可顺利完成。称 <P1、P2、P3.....Pn> 序列为安全序列。

我们对线程 2 的代码修改成下面这样就不会产生死锁了。

```
new Thread(() -> {
    synchronized (resource1) {
        System.out.println(Thread.currentThread() + "get
resource1");

        try {
            Thread.sleep(1000);
```

```
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.println(Thread.currentThread() + "waiting get  
resource2");  
        synchronized (resource2) {  
            System.out.println(Thread.currentThread() + "get  
resource2");  
        }  
    }  
}, "线程 2").start();
```

输出：

```
Thread[线程 1,5,main]get resource1  
Thread[线程 1,5,main]waiting get resource2  
Thread[线程 1,5,main]get resource2  
Thread[线程 2,5,main]get resource1  
Thread[线程 2,5,main]waiting get resource2  
Thread[线程 2,5,main]get resource2
```

我们分析一下上面的代码为什么避免了死锁的发生？

线程 1 首先获得到 resource1 的监视器锁,这时候线程 2 就获取不到了。然后线程 1 再去获取 resource2 的监视器锁，可以获取到。然后线程 1 释放了对 resource1、resource2 的监视器锁的占用，线程 2 获取到就可以执行了。这样就破坏了破坏循环等待条件，因此避免了死锁。

摘自：[Java 并发常见面试题总结（上）](#)。

JavaGuide官方知识星球 (限时优惠)

专属面试小册/一对一提问/简历修改
专属求职指南/不定时福利/学习打卡

— 点击图片即可详细了解 —

快速排序

基本概念：

八大常见排序算法之一，也是面试必须要掌握的，小伙伴们应该都会写，所以这里就简单提及一下，如果忘了可以再回到这里来温习一下。

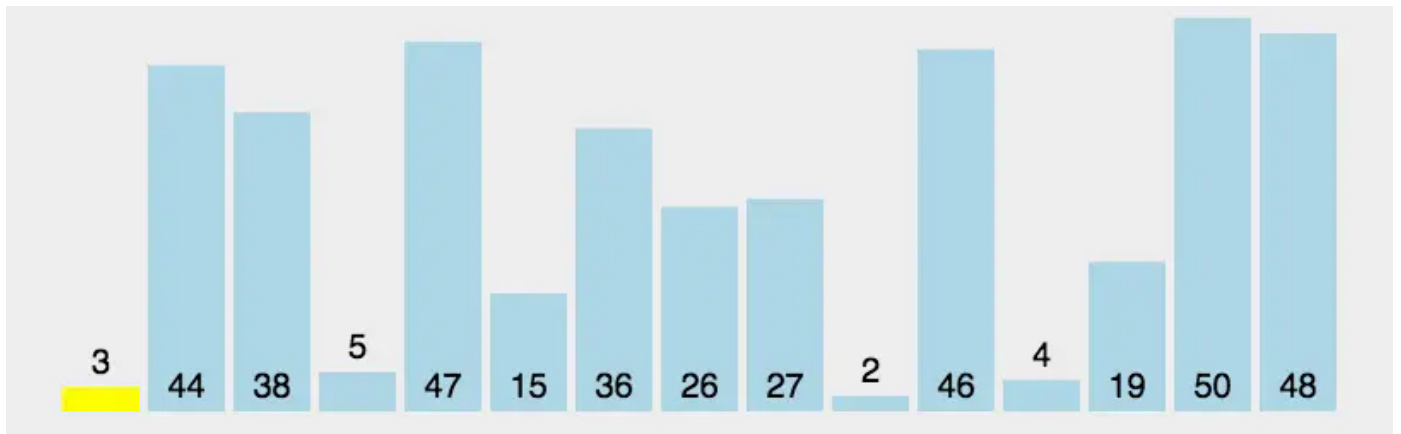
快速排序是一种基于分治思想的排序算法。它的基本思想是选择一个基准元素，通过一趟排序将数组划分为两个子数组，其中一个子数组的所有元素都小于基准元素，另一个子数组的所有元素都大于基准元素。然后对这两个子数组分别递归地应用快速排序算法，直到整个数组有序。

快速排序的平均时间复杂度为： $O(n\log n)$ ，然而，最坏情况下快速排序的时间复杂度为 $O(n^2)$ ，即当数组已经有序（或基本有序）时，快速排序的效率会退化到最差情况。为了避免最坏情况的发生，一种常见的优化方法是随机选择基准元素，以减小最坏情况发生的概率。

快速排序属于比较排序，它的兄弟们还有：归并排序、堆排序、冒泡排序；在排序的最终结果里，元素之间的次序依赖于它们之间的比较。每个数都必须和其他数进行比较，才能确定自己的位置。

比较排序的优势是：适用于各种规模的数据，也不在乎数据的分布，都能进行排序。可以说，比较排序适用于一切需要排序的情况。

上面讲了这么多，下面进入实操，先来看一下演示图：



上述步骤：

1. 首先，我们选择一个**基准元素**（比如数组中的一个数）。
2. 接下来，我们将整个数组分成两部分，一部分是比**基准元素**小的数，另一部分是比基准元素大的数。
3. 然后，我们分别对这两部分进行循环操作，重复上述步骤，直到每部分只有一个元素或为空。
4. 最后，我们将所有的部分合并起来，就得到了一个有序的数组。

代码如下：

```
public class QuickSort {  
    public static void main(String[] args) {  
        int[] arr = {5, 9, 3, 1, 8, 6, 4, 2, 7};  
        quickSort(arr, 0, arr.length - 1);  
  
        System.out.println("排序结果：");  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
    }  
  
    public static void quickSort(int[] arr, int low, int high) {  
        if (low < high) {  
            int pivot = partition(arr, low, high); // 将数组划分为两部分
```

```

        // 分别对左右子数组进行递归排序
        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[low]; // 选择第一个元素作为基准元素
    int i = low, j = high + 1;

    while (true) {
        // 从左往右找第一个大于等于基准元素的数
        while (arr[++i] < pivot) {
            if (i == high) {
                break;
            }
        }
        // 从右往左找第一个小于等于基准元素的数
        while (arr[--j] > pivot) {
            if (j == low) {
                break;
            }
        }

        if (i >= j) {
            break;
        }

        // 交换左右两个数
        swap(arr, i, j);
    }

    // 将基准元素放到正确的位置上
    swap(arr, low, j);
}

```



```
        return j; // 返回基准元素的索引
    }

    public static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

以上是一个简单的快速排序的 Java 实现。在示例中，我们使用 `quickSort` 方法进行递归排序，并通过 `partition` 方法将数组划分为两部分。交换元素的操作由 `swap` 方法完成。

更多排序算法的代码实现和讲解，推荐阅读 [十大经典排序算法总结](#) 这篇文章。

生产者和消费者

在 Java 并发中，这两个东西是经常出现在我们视野中，那么让我们再来回顾一下它们之间的概念吧！

生产者和消费者是指两个并发执行的程序或线程，其中一个程序或线程生成一些数据（称为“生产”），而另一个程序或线程使用这些数据（称为“消费”）。

举例说明： 想象一下你去超市购物的场景。超市有一些商品（数据），而你是购物者（消费者）。超市的员工把商品放在货架上（生产者），你可以从货架上拿走需要的商品。生产者和消费者模型就是描述这种角色分工的一种模型。

在计算机程序中，也有类似的情况。生产者就像一个程序或者线程，它生成一些数据。而消费者则是另外一个程序或者线程，它使用这些数据进行一些操作。

但是，在多线程编程中，有个问题需要解决：如何确保生产者不会往满了的数据缓冲区继续添加数据，而消费者也不会从空的缓冲区取数据。为了解决这个问题，我们需要引入一个共享的缓冲区，生产者把数据放到缓冲区里，而消费者从缓冲区中取出数据。这样，生产者和消费者之间就能协调好彼此的操作，避免冲突。

生产者和消费者模型被广泛应用在实际编程中，比如操作系统控制进程间的共享资源、多线程程序中的线程协作等等。它帮助我们解决并发问题，确保不同线程之间能够安全地共享和操作数据。

生产者：

```
import java.util.concurrent.BlockingQueue;

class Producer implements Runnable {
    private final BlockingQueue<Integer> queue;

    public Producer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        try {
            for (int i = 1; i <= 10; i++) {
                // 生产物品
                produce(i);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void produce(int item) throws InterruptedException {
        // 将物品放入缓冲区
        queue.put(item);

        System.out.println("生产者生产物品: " + item + ", 队列大小: " +
            queue.size());
    }
}
```

消费者:

```
import java.util.concurrent.BlockingQueue;

class Consumer implements Runnable {
    private final BlockingQueue<Integer> queue;

    public Consumer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        try {
            while (true) {
                // 消费物品
                consume();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void consume() throws InterruptedException {
        // 从缓冲区取出物品
        int item = queue.take();
        System.out.println("消费者消费物品: " + item + ", 队列大小: " +
            queue.size());
    }
}
```

测试代码:

```
// 创建大小为 5 的缓冲区
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);
// 创建生产者和消费者
Producer producer = new Producer(queue);
Consumer consumer = new Consumer(queue);

Thread producerThread = new Thread(producer);
Thread consumerThread = new Thread(consumer);

// 启动生产者和消费者线程
producerThread.start();
consumerThread.start();
```

输出：

```
生产者生产物品：1，队列大小：1
生产者生产物品：2，队列大小：2
生产者生产物品：3，队列大小：3
生产者生产物品：4，队列大小：4
生产者生产物品：5，队列大小：5
生产者生产物品：6，队列大小：5
消费者消费物品：1，队列大小：5
消费者消费物品：2，队列大小：4
消费者消费物品：3，队列大小：4
生产者生产物品：7，队列大小：5
消费者消费物品：4，队列大小：3
生产者生产物品：8，队列大小：4
消费者消费物品：5，队列大小：3
生产者生产物品：9，队列大小：4
消费者消费物品：6，队列大小：3
生产者生产物品：10，队列大小：4
消费者消费物品：7，队列大小：3
消费者消费物品：8，队列大小：2
```

消费者消费物品：9，队列大小：1

消费者消费物品：10，队列大小：0

在上述代码中生产者和消费者共享一个阻塞队列 `BlockingQueue` 作为缓冲区。当缓冲区已满时，生产者线程等待，并在队列可用时将物品放入缓冲区。当缓冲区为空时，消费者线程等待，并在队列中有物品时从缓冲区中取出物品。

⚠ 注意： `BlockingQueue` 自身已经处理了线程安全和阻塞等待，因此不需要额外的同步机制。

阻塞队列

具体答案可以参考一位球友的分享：<https://t.zsxq.com/18eBSjxJo>。

普通队列实现：

阻塞队列其实本质上也是一个队列，所以我们需要先把队列实现出来，主要是基于数组实现循环队列，这里就不做过多赘述了，代码如下：

```
class BlockingQueue{
    private int[] items;
    private int head = 0; //队列头指针
    private int tail = 0; //队列尾指针
    private int size = 0; //队列当前元素个数
    public BlockingQueue(){}
    //入队列
    public void put(int elem){}
    //出队列
    public int take(){}
}
```

构造方法：

```
public BlockingQueue(){  
    this.items = new int[100];  
}
```

入队列:

```
public void put(int elem){  
    if (size >= items.length){  
        return;  
    }  
    if (tail >= items.length){//判断尾指针是否到达末尾  
        tail = 0;  
    }  
    items[tail] = elem;  
    tail++;  
    size++;  
}
```

出队列:

```

public int take(){
    if(size == 0){
        return -1;
    }
    if (head >= items.length){
        head = 0;
    }
    int elem = items[head];
    head++;
    size--;
    return elem;
}

```

阻塞队列实现：

然后在原先的基础上，就可以进行阻塞队列的改造了，首先，我们要实现阻塞队列，第一个要考虑的点就是线程安全，第二个点就是阻塞的改造

1. 线程安全这个点我们就是加一个 `synchronized` 的关键字即可
2. 然后由于我们采用的是阻塞的方式，只需要将原先的 `return` 改造成等待即可，且当元素出队和入队之后，需要唤醒当前因为队列满而被阻塞的线程状态。

```

public void put(int elem) throws InterruptedException {
    synchronized (this){
        while(size >= items.length){
            // 队列满了，采取等待策略
            this.wait();
        }
        // 队列没满：添加元素
        if(tail >= items.length ){
            tail = 0;
        }
        items[tail] = elem;
    }
}

```

```

        tail++;
        size++;
        // 成功入队，唤醒因为队列空间被阻塞进入的状态
        this.notify();
    }
}

```

get 方法和上面一样，进行一个对应的改造即可：

```

/**
 * 出队列
 * @return
 * @throws InterruptedException
 */
public int take() throws InterruptedException {
    synchronized (this){
        while (size == 0){
            //队列空
            //return -1;
            this.wait();
        }
        if (head >= items.length){
            head = 0;
        }
        int elem = items[head];

        head++;
        size--;
        this.notify();//使用这个notify唤醒队列满的阻塞状态
        return elem;
    }
}

```


内存可见性：

然后在上面保证线程安全之后，我们还需要考虑一个重要的点，就是内存可见性，即在多线程的情况，不光是对变量进行修改，还有读操作等等，那就有可能出现一个线程在读，另外一个线程在修改，这个读的线程没有读到。所以，此处除了加锁之外，还需要考虑内存可见性问题。也就是说，当其他线程进行修改的时候，我们要保证当前线程可以读到这个修改，所以我们将变量加上 `volatile` 关键字。

```
volatile private int head = 0;
volatile private int tail = 0;
volatile private int size = 0;
```

最终代码：

```
public class BlockingQueue {
    private int[] items;
    volatile private int head = 0;
    volatile private int tail = 0;
    volatile private int size = 0;
    public BlockingQueue(){
        this.items = new int[100];
    }

    /**
     * 手撕阻塞队列——入队
     * @param elem
     * @throws InterruptedException
     */
    public void put(int elem) throws InterruptedException {
        synchronized (this){
            while(size >= items.length){
                // 队列满了，采取等待策略
                this.wait();
            }
        }
    }
}
```

```

    }

    // 队列没满:添加元素
    if(tail >= items.length ){
        tail = 0;
    }

    items[tail] = elem;
    tail++;
    size++;
    // 成功入队，唤醒因为队列空间被阻塞进入的状态
    this.notify();
}

}

/**
 * 出队列
 * @return
 * @throws InterruptedException
 */
public int take() throws InterruptedException {
    synchronized (this){
        while (size == 0){
            //队列空
            //return -1;
            this.wait();
        }

        if (head >= items.length){
            head = 0;
        }

        int elem = items[head];
        head++;
        size--;
        this.notify();//使用这个notify唤醒队列满的阻塞状态
        return elem;
    }
}

```

```

public static void main(String[] args) {
    BlockingQueue queue = new BlockingQueue();
    //生产者线程
    Thread product = new Thread()->{
        int count = 0;
        while (true){
            try {
                queue.put(count);
                System.out.println("生产元素:>"+count);
                count++;
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });
    //消费者线程
    Thread consumer = new Thread()->{
        while (true){
            try {
                int elem = queue.take();
                System.out.println("消费元素:>"+elem);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });
    product.start();
    consumer.start();
}
}

```

建议

最后，在笔试过程中，以下是一些简单的步骤和小建议，供大家参考。

1. **理解题目要求：**认真阅读题目，确保完全理解问题的要求和限制条件。理解问题的关键点对于解决问题至关重要。
2. **分析问题：**思考问题的解决方法和可能的算法或数据结构。确定问题是属于哪个领域，例如搜索、排序、图论等，并尝试找到解决问题的相关技巧和方法。
3. **设计算法：**根据问题的要求，设计一个合适的算法来解决问题。考虑使用哪些数据结构、循环或递归等技术。
4. **编写代码：**将算法转化为实际的代码。在编写代码时，遵循编码规范，使用清晰的变量和函数命名，并添加必要的注释以提高可读性。
5. **优化和改进：**如果代码能够正确执行，但效率不高或计算时间过长，可以思考优化算法的方式，以达到更好的性能和效果。尝试使用更有效的数据结构或算法。
6. **时间管理：**在笔试过程中，时间管理非常重要。根据题目的难度和分值，合理安排时间来解决问题。如果遇到困难的问题，可以跳过并在之后回来解决。
7. **思维明晰：**保持冷静和清晰的思维对于解决问题至关重要。当遇到困难时，不要感到沮丧或紧张，尽量放松心情，越紧张越急！
8. **实践和准备：**在实际练习中解决各种类型的问题，熟悉常见的数据结构和算法，积累经验和技巧。这将有助于在笔试过程中更自信和专注地解决问题。

当然，在面试前提前准备并进行大量的练习也是非常重要的。这将帮助你在笔试过程中更加熟悉和自信地应对各种类型的问题。同时要熟读面经，多看看其他人遇到的问题，然后自己尝试着去解决和分析问题，并且“情景代入”，试问自己如果在某天面试中碰到此类问题，自己的是否有思路、有能力去成功解决；在引入了自己的独立思考后，即使没有做出来，印象将大大加深，以后再碰到同类问题也就迎刃而解了，希望大家能在笔试中取得好成绩，成功进面。

祝大家面试顺利！！！！